

Piscine Reloaded

It's good to be back

Summary:
The Piscine was good but the time has passed. This series of exercises will help you remember all the basics you've learned during the piscine. Functions, loops, pointers, structures, let's review together the syntactic and semantic basics of the `C`

Version: 2.2

Contents

I	Foreword	3
II	Introduction	4
III	General rules	5
IV	AI Instructions	7
V	Exercise 00 : Oh yeah, mooore...	9
VI	Exercise 01 : Z	10
VII	Exercise 02 : clean	11
VIII	Exercise 03 : find_sh	12
IX	Exercise 04 : MAC	13
X	Exercise 05 : Can you create it ?	14
XI	Exercise 06 : ft_print_alphabet	15
XII	Exercise 07 : ft_print_numbers	16
XIII	Exercise 08 : ft_is_negative	17
XIV	Exercise 09 : ft_ft	18
XV	Exercise 10 : ft_swap	19
XVI	Exercise 11 : ft_div_mod	20
XVII	Exercise 12 : ft_iterative_factorial	21
XVIII	Exercise 13 : ft_recursive_factorial	22
XIX	Exercise 14 : ft_sqrt	23
XX	Exercise 15 : ft_putstr	24
XXI	Exercise 16 : ft_strlen	25
XXII	Exercise 17 : ft_strcmp	26
XXIII	Exercise 18 : ft_print_params	27

XXIV Exercise 19 : ft_sort_params **28**

XXV Exercise 20 : ft_strdup **29**

XXVI Exercise 21 : ft_range **30**

XXVII Exercise 22 : ft_abs.h **31**

XXVIII Exercise 23 : ft_point.h **32**

XXIX Exercise 24 : Makefile **33**

XXX Exercise 25 : ft_foreach **34**

XXXI Exercise 26 : ft_count_if **35**

XXXII Exercise 27 : display_file **36**

XXXIII Submission and peer-evaluation **37**

Chapter I

Foreword

Edward Joseph Snowden (born June 21, 1983) is an American computer professional, former Central Intelligence Agency (CIA) employee, and former contractor for the United States government who copied and leaked classified information from the National Security Agency (NSA) in 2013 without authorization. His disclosures revealed numerous global surveillance programs, many run by the NSA and the Five Eyes Intelligence Alliance with the cooperation of telecommunication companies and European governments.

In 2013, Snowden was hired by an NSA contractor, Booz Allen Hamilton, after previous employment with Dell and the CIA. On May 20, 2013, Snowden flew to Hong Kong after leaving his job at an NSA facility in Hawaii, and in early June he revealed thousands of classified NSA documents to journalists Glenn Greenwald, Laura Poitras, and Ewen MacAskill. Snowden came to international attention after stories based on the material appeared in The Guardian and The Washington Post. Further disclosures were made by other publications including Der Spiegel and The New York Times.

On June 21, 2013, the U.S. Department of Justice unsealed charges against Snowden of two counts of violating the Espionage Act of 1917 and theft of government property. Two days later, he flew into Moscow's Sheremetyevo Airport, but Russian authorities noted that his U.S. passport had been cancelled and he was restricted to the airport terminal for over one month. Russia ultimately granted him right of asylum for one year, and repeated extensions have permitted him to stay at least until 2020. He reportedly lives in an undisclosed location in Moscow, and continues to seek asylum elsewhere in the world.

A subject of controversy, Snowden has been variously called a hero, a whistleblower, a dissident, a traitor and a patriot. His disclosures have fueled debates over mass surveillance, government secrecy, and the balance between national security and information privacy.

If you'd like to find out more, we recommend you watch the documentary `Citizenfour`.

Chapter II

Introduction

The `Piscine Reloaded` is a best-of of the exercises you did during the `C Piscine` to remind you of all the basics of the `C` programming language.

If you have already done some of these exercises during the C Piscine, we strongly recommend that you not be tempted to reuse your old code. Learning to program takes practice, and reusing existing code teaches you nothing.

Chapter III

General rules

- Only this page will serve as reference; do not trust rumors.
- Make sure you have the appropriate permissions on your files and directories.
- You have to follow the turn-in procedures for every exercise.
- Your exercises will **only** be checked and graded by a program called Moulinette.
- Moulinette is very meticulous and strict in its evaluation of your work. It is entirely automated and there is no way to negotiate with it. So if you want to avoid bad surprises, be as thorough as possible.
- Exercises in Shell must be executable with /bin/sh.
- You cannot leave any additional file in your directory than those specified in the subject.
- Got a question? Ask your peer on the right. Otherwise, try your peer on the left.
- Your reference guide is called `Google / man / the Internet / ...`.
- Examine the examples thoroughly. They could very well call for details that are not explicitly mentioned in the subject...
- Moulinette is not very open-minded. It won't try and understand your code if it doesn't respect the Norm. Moulinette relies on a program called `norminette` to check if your files respect the norm. TL;DR: it would be idiotic to submit a piece of work that doesn't pass `norminette`'s check.
- Using a forbidden function is considered cheating. Cheaters get `-42`, and this grade is non-negotiable.
- You'll only have to submit a main() function if we ask for a program.
- Moulinette compiles with these flags: -Wall -Wextra -Werror, and uses `CC`.
- If ft_putchar() is an authorized function, we will compile your code with our `ft_putchar.c`.
- If your program doesn't compile, you'll get `0`.

- By Odin, by Thor! Use your brain!!!

Chapter IV

AI Instructions

Context

This project is designed to help you discover the fundamental building blocks of your 42 training.

To properly anchor key knowledge and skills, it's essential to adopt a thoughtful approach to using AI tools and support.

True foundational learning requires genuine intellectual effort — through challenge, repetition, and peer-learning exchanges.

For a more complete overview of our stance on AI — as a learning tool, as part of the 42 training, and as an expectation in the job market — please refer to the dedicated FAQ on the intranet.

Main message

- Build strong foundations without shortcuts.
- Really develop tech & power skills.
- Experience real peer-learning, start learning how to learn and solve new problems.
- The learning journey is more important than the result.
- Learn about the risks associated with AI, and develop effective control practices and countermeasures to avoid common pitfalls.

Learner rules:

- You should apply reasoning to your assigned tasks, especially before turning to AI.

- You should not ask for direct answers to the AI.
- You should learn about 42 global approach on AI.

Phase outcomes:

Within this foundational phase, you will get the following outcomes:

- Get proper tech and coding foundations.
- Know why and how AI can be dangerous during this phase.

Comments and example:

- Yes, we know AI exists — and yes, it can solve your projects. But you're here to learn, not to prove that AI has learned. Don't waste your time (or ours) just to demonstrate that AI can solve the given problem.
- Learning at 42 isn't about knowing the answer — it's about developing the ability to find one. AI gives you the answer directly, but that prevents you from building your own reasoning. And reasoning takes time, effort, and involves failure. The path to success is not supposed to be easy.
- Keep in mind that during exams, AI is not available — no internet, no smartphones, etc. You'll quickly realise if you've relied too heavily on AI in your learning process.
- Peer learning exposes you to different ideas and approaches, improving your interpersonal skills and your ability to think divergently. That's far more valuable than just chatting with a bot. So don't be shy — talk, ask questions, and learn together!
- Yes, AI will be part of the curriculum — both as a learning tool and as a topic in itself. You'll even have the chance to build your own AI software. In order to learn more about our crescendo approach you'll go through in the documentation available on the intranet.

✓ Good practice:

> I'm stuck on a new concept. I ask someone nearby how they approached it. We talk for 10 minutes — and suddenly it clicks. I get it.

✗ Bad practice:

> I secretly use AI, copy some code that looks right. During peer evaluation, I can't explain anything. I fail. During the exam — no AI — I'm stuck again. I fail.

Chapter V

Exercise 00 : Oh yeah, mooore...

	Exercise 00
Oh yeah, mooore...	
Directory: *ex*00/	
Files to Submit: `exo.tar`	
Authorized: `None`	

- Create the following files and directories. Do what's necessary so that when you use the `ls -l` command in your directory, the output will look like this:

```
%> ls -l
total XX
drwx--xr-x 2 XX XX  XX Jun 1 20:47 test0
-rwx--xr-- 1 XX XX   4 Jun 1 21:46 test1
dr-x---r-- 2 XX XX  XX Jun 1 22:45 test2
-r-----r-- 2 XX XX   1 Jun 1 23:44 test3
-rw-r----x 1 XX XX   2 Jun 1 23:43 test4
-r-----r-- 2 XX XX   1 Jun 1 23:44 test5
lrwxrwxrwx 1 XX XX   5 Jun 1 22:20 test6 -> test0
%>
```

- About the hours, it will be accepted if the year is diplayed in the case of the exercise's date (1 Jun) is outdated by six month or more.
- Once you've done that, run `tar -cf exo.tar *` to create the file to be submitted.



`Don't worry about what you've got instead of "XX".`

Chapter VI

Exercise 01 : Z

	Exercise 01
Only the best know how to display Z	
Directory: *ex*01/	
Files to Submit: `z`	
Authorized: `None`	

- Create a file called `z` that returns "Z", followed by a new line, whenever the command `cat` is used on it.

```
?>cat z
Z
?>
```

Chapter VII

Exercise 02 : clean

	Exercise 02
Directory: *ex*02/	
Files to Submit: `clean`	
Authorized: `None`	

- In a file called `clean`, place the command line that will search the current directory and all its sub-directories for files whose name ends with ~, or starts and ends with `#`
- The command line will show and erase all files found.
- Only one command is allowed: no ';' or '&&' or other shenanigans.



```
man find
```

Chapter VIII

Exercise 03 : find_sh

	Exercise 03
find_sh.sh	
Directory: *ex*03/	
Files to Submit: `find_sh.sh`	
Authorized: `None`	

- Write a command line that searches for all file names that end with ".sh" (without quotation marks) in the current directory and all its sub-directories. It should display only the file names without the .sh.
- Example of output :

```
$>./find_sh.sh | cat -e
find_sh$
file1$
file2$
file3$
$>
```

Chapter IX

Exercise 04 : MAC

	Exercise 04
MAC.sh	
Directory: *ex*04/	
Files to Submit: `MAC.sh`	
Authorized: `None`	

- Write a command line that displays your machine's MAC addresses. Each address must be followed by a line break.



```
man ifconfig
```

Chapter X

Exercise 05 : Can you create it ?

	Exercise 05
Can you create it ?	
Directory: *ex*05/	
Files to Submit: `"\?$*'MaRViN'*$?\"`	
Authorized: `None`	

- Create a file containing only "42", and NOTHING else.
- Its name will be :

```
"\?$*'MaRViN'*$?\"
```

- Example :

```
$>ls -lRa *MaRV* | cat -e
-rw---xr-- 1 75355 32015 2 Oct 2 12:21 "\?$*'MaRViN'*$?\"$
$>
```

Chapter XI

Exercise 06 : ft_print_alphabet

	Exercise 06
ft_print_alphabet	
Directory: *ex*06/	
Files to Submit: `ft_print_alphabet.c`	
Authorized: `ft_putchar`	

- Create a function that displays the alphabet in lowercase, on a single line, by ascending order, starting from the letter 'a'.
- Here's how it should be prototyped :

```
void ft_print_alphabet(void);
```

Chapter XII

Exercise 07 : ft_print_numbers

	Exercise 07
ft_print_numbers	
Directory: *ex*07/	
Files to Submit: `ft_print_numbers.c`	
Authorized: `ft_putchar`	

- Create a function that displays all digits, on a single line, by ascending order.
- Here's how it should be prototyped :

```
void ft_print_numbers(void);
```

Chapter XIII

Exercise 08 : ft_is_negative

	Exercise 08
ft_is_negative	
Directory: *ex*08/	
Files to Submit: `ft_is_negative.c`	
Authorized: `ft_putchar`	

- Create a function that displays 'N' or 'P' depending on the integer's sign entered as a parameter. If `n` is negative, display 'N'. If `n` is positive or null, display 'P'.
- Here's how it should be prototyped :

```
void ft_is_negative(int n);
```

Chapter XIV

Exercise 09 : ft_ft

	Exercise 09
ft_ft	
Directory: *ex*09/	
Files to Submit: `ft_ft.c`	
Authorized: `None`	

- Create a function that takes a pointer to int as a parameter, and sets the value "42" to that int.
- Here's how it should be prototyped :

```
void  ft_ft(int *nbr);
```

Chapter XV

Exercise 10 : ft_swap

	Exercise 10
ft_swap	
Directory: *ex*10/	
Files to Submit: `ft_swap.c`	
Authorized: `None`	

- Create a function that swaps the value of two integers whose addresses are entered as parameters.
- Here's how it should be prototyped :

```
void    ft_swap(int *a, int *b);
```

Chapter XVI

Exercise 11 : ft_div_mod

	Exercise 11
ft_div_mod	
Directory: *ex*11/	
Files to Submit: `ft_div_mod.c`	
Authorized: `None`	

- Create a function `ft_div_mod` prototyped like this :

```
void    ft_div_mod(int a, int b, int *div, int *mod);
```

- This function divides parameters `a` by `b` and stores the result in the int pointed by `div`. It also stores the remainder of the division of `a` by `b` in the int pointed by `mod`.

Chapter XVII

Exercise 12 : ft_iterative_factorial

	Exercise 12
ft_iterative_factorial	
Directory: *ex*12/	
Files to Submit: `ft_iterative_factorial.c`	
Authorized: `None`	

- Create an iterated function that returns a number. This number is the result of a factorial operation based on the number given as a parameter.
- If there's an error, the function should return 0.
- Here's how it should be prototyped :

```
int ft_iterative_factorial(int nb);
```

- Your function must return its result in less than two seconds.

Chapter XVIII

Exercise 13 : ft_recursive_factorial

	Exercise 13
ft_recursive_factorial	
Directory: *ex*13/	
Files to Submit: `ft_recursive_factorial.c`	
Authorized: `None`	

- Create a recursive function that returns the factorial of the number given as a parameter.
- If there's an error, the function should return 0.
- Here's how it should be prototyped :

```
int ft_recursive_factorial(int nb);
```

Chapter XIX

Exercise 14 : ft_sqrt

	Exercise 14
ft_sqrt	
Directory: *ex*14/	
Files to Submit: `ft_sqrt.c`	
Authorized: `None`	

- Create a function that returns the square root of a number (if it exists), or 0 if the square root is an irrational number.
- Here's how it should be prototyped :

```
int ft_sqrt(int nb);
```

- Your function must return its result in less than two seconds.

Chapter XX

Exercise 15 : ft_putstr

	Exercise 15
ft_putstr	
Directory: $ex15/$	
Files to Submit: `ft_putstr.c`	
Authorized: `ft_putchar`	

- Create a function that displays a string of characters on the standard output.
- Here's how it should be prototyped :

```
void    ft_putstr(char *str);
```

Chapter XXI

Exercise 16 : ft_strlen

	Exercise 16
ft_strlen	
Directory: *ex*16/	
Files to Submit: `ft_strlen.c`	
Authorized: `None`	

- Reproduce the behavior of the function`strlen` (man strlen).
- Here's how it should be prototyped :

```
int ft_strlen(char *str);
```

Chapter XXII

Exercise 17 : ft_strcmp

	Exercise 17
ft_strcmp	
Directory: *ex*17/	
Files to Submit: `ft_strcmp.c`	
Authorized: `None`	

- Reproduce the behavior of the function`strcmp` (man strcmp).
- Here's how it should be prototyped :

```
int ft_strcmp(char *s1, char *s2);
```

Chapter XXIII

Exercise 18 : ft_print_params

	Exercise 18
ft_print_params	
Directory: *ex*18/	
Files to Submit: `ft_print_params.c`	
Authorized: `ft_putchar`	

- We're dealing with a program here, you should therefore have a function `main` in your `.c` file.
- Create a program that displays its given arguments.
- Example :

```
$>./a.out test1 test2 test3
test1
test2
test3
$>
```

Chapter XXIV

Exercise 19 : ft_sort_params

	Exercise 19
ft_sort_params	
Directory: *ex*19/	
Files to Submit: `ft_sort_params.c`	
Authorized: `ft_putchar`	

- We're dealing with a program here, you should therefore have a function `main` in your `.c` file.
- Create a program that displays its given arguments sorted by ascii order.
- It should display all arguments, except for `argv[0]`.
- All arguments have to have their own line.

Chapter XXV

Exercise 20 : ft_strdup

	Exercise 20
ft_strdup	
Directory: *ex*20/	
Files to Submit: `ft_strdup.c`	
Authorized: `malloc`	

- Reproduce the behavior of the function`strdup` (man strdup).
- Here's how it should be prototyped :

```
char    *ft_strdup(char *src);
```

Chapter XXVI

Exercise 21 : ft_range

	Exercise 21
ft_range	
Directory: *ex*21/	
Files to Submit: `ft_range.c`	
Authorized: `malloc`	

- Create a function `ft_range` which returns an array of`ints`. This `int` array should contain all values between `min` and `max`.
- `Min` included - `max` excluded.
- Here's how it should be prototyped :

```
int    *ft_range(int min, int max);
```

- If `min`́value is greater or equal to `max`'s value, a null pointer should be returned.

Chapter XXVII

Exercise 22 : ft_abs.h

	Exercise 22
ft_abs.h	
Directory: *ex*22/	
Files to Submit: `ft_abs.h`	
Authorized: `None`	

- Create a `macro` `ABS` which replaces its argument by it absolute value :

```
#define ABS(Value)
```



You are asked to do something that is normally banned by the Norm, that will be the only time we autorize it.

Chapter XXVIII

Exercise 23 : ft_point.h

	Exercise 23
ft_point.h	
Directory: *ex23/*	
Files to Submit: `ft_point.h`	
Authorized: `None`	

- Create a file `ft_point.h` that'll compile the following main :

```c
#include "ft_point.h"

void  set_point(t_point *point)
{
  point->x = 42;
  point->y = 21;
}

int   main(void)
{
  t_point   point;

  set_point(&point);
  return (0);
}
```

Chapter XXIX

Exercise 24 : Makefile

	Exercise 24
Makefile	
Directory: *ex*24/	
Files to Submit: `Makefile`	
Authorized: `None`	

- Create the `Makefile` that'll compile your `libft.a`.
- The `Makefile` will get its source files from the "srcs" directory.
- The `Makefile` will get its header files from the "includes" directory.
- The lib will be at the root of the exercise.
- The `Makefile` should also implement the following rules: `clean`, `fclean` and `re` as well as `all`.
- `fclean` does the equivalent of a make clean and also erases the binary created during the make. `re` does the equivalent of a make fclean followed by a make.
- We'll only fetch your Makefile and test it with our files. For this exercise, only the following 5 mandatory functions of your lib have to be handled : (`ft_putchar`, `ft_putstr`, `ft_strcmp`, `ft_strlen` and `ft_swap`).



`Watch out for wildcards!`

Chapter XXX

Exercise 25 : ft_foreach

	Exercise 25
ft_foreach	
Directory: *ex*25/	
Files to Submit: `ft_foreach.c`	
Authorized: `None`	

- Create the function `ft_foreach` which, for a given ints array, applies a function on all elements of the array. This function will be applied following the array's order.
- Here's how the function should be prototyped :

```
void  ft_foreach(int *tab, int length, void (*f)(int));
```

- For example, the function `ft_foreach` could be called as follows in order to display all ints of the array :

```
ft_foreach(tab, 1337, &ft_putnbr);
```

Chapter XXXI

Exercise 26 : ft_count_if

	Exercise 26
ft_count_if	
Directory: *ex*26/	
Files to Submit: `ft_count_if.c`	
Authorized: `None`	

- Create a function `ft_count_if` which will return the number of elements of the array that return 1, passed to the function `f`.
- Here's how the function should be prototyped :

```
int   ft_count_if(char **tab, int (*f)(char*));
```

- The array will be terminated by 0.

Chapter XXXII

Exercise 27 : display_file

	Exercise 27
display_file	
Directory: *ex*27/	
Files to Submit: `Makefile, and files needed for your program`	
Authorized: `close, open, read, write`	

- Create a program called `ft_display_file` that displays, on the standard output, only the content of the file given as argument.
- The submission directory should have a `Makefile` with the following rules : `all`, `clean`, `fclean`. The binary will be called `ft_display_file`.
- The `malloc` function is forbidden. You can only do this exercise by declaring a fixed-sized array.
- All files given as arguments will be valid.
- Error messages have to be displayed on their reserved output followed by a new line.
- If no argument is given, it should display

```
File name missing.
```

- If there is more than one argument, it should display

```
Too many arguments.
```

- If the file cannot be read, it should display

```
Cannot read file.
```

Chapter XXXIII

Submission and peer-evaluation

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Don't hesitate to double check the names of your folders and files to ensure they are correct.